Course lecture notes

Machine Intelligence I Lecture Notes

Complete course synthesis with equations and intuition

A LaTeX study guide synthesizing the full Machine Intelligence I lecture sequence from supervised learning foundations through Bayesian inference and reinforcement learning.

Coverage

- Learning paradigms, connectionist neurons, multilayer perceptrons, and optimization
- Deep learning, recurrent neural networks, and radial basis function networks
- Statistical learning theory and support vector machines
- Uncertainty, Bayesian networks, Bayesian model selection, and Bayesian prediction
- Reinforcement learning: value functions, Q-values, policy iteration, and exploration

Prepared for Arjun Puri

Generated in LaTeX with Bodhi • April 15, 2026

Contents

Orientation **1**

1 Introduction and learning paradigms **1**
1.1 Learning as model selection . . . 1
1.2 The three main paradigms . . . 2
1.3 What matters conceptually . . . 2

2 Connectionist neurons **2**
2.1 Two key intuitions . . . 2
2.2 Transfer functions . . . 3
2.3 Geometry to remember . . . 3

3 Multilayer perceptrons and backpropagation **3**
3.1 Network classes . . . 3
3.2 Universal approximation — with a caveat . . . 4
3.3 Backpropagation in one sentence . . . 4
3.4 Training loop . . . 4
3.5 Classification link . . . 4

4 Optimization, regularization, and validation **4**
4.1 Optimization regimes . . . 5
4.1.1 Batch gradient descent . . . 5
4.1.2 Online / stochastic gradient descent . . . 5
4.1.3 Mini-batch optimization . . . 5
4.2 Improving optimization . . . 5
4.3 Training error vs. generalization error . . . 5
4.4 Overfitting, underfitting, bias, variance . . . 6
4.5 Regularization tools . . . 6

5 Deep learning foundations **6**
5.1 Why deep networks originally looked difficult . . . 6
5.2 Why deep learning came back . . . 7
5.3 Deep representations . . . 7
5.4 Autoencoders and pre-training . . . 7
5.5 Practical regularization in deep networks . . . 7

6 Recurrent neural networks **8**
6.1 Core RNN dynamics . . . 8
6.2 Backpropagation through time . . . 8
6.3 Why long-term dependencies are hard . . . 8
6.4 Architectural fixes . . . 9

7 Radial basis function networks **9**
7.1 Why RBFs are attractive . . . 9
7.2 Two-step learning procedure . . . 9
7.3 Closed-form output learning . . . 10
7.4 Normalization and regularization . . . 10

8 Statistical learning theory **10**
8.1 Capacity and the growth function . . . 11
8.2 VC dimension . . . 11
8.3 Structural risk minimization . . . 11

9 Support vector machines **11**
9.1 Hard-margin primal problem . . . 12
9.2 Dual formulation and support vectors . . . 12
9.3 Soft margins and kernels . . . 12

10 Uncertainty and inference **13**
10.1 Inference by marginalization and conditioning . . . 13
10.2 Conditional independence . . . 13

11 Bayesian networks **14**
11.1 Graph structure and independence . . . 14
11.2 Markov blankets . . . 14
11.3 Inference in Bayesian networks . . . 14

12 Bayesian inference and neural networks **14**
12.1 Bayesian model selection . . . 15
12.2 Bayesian prediction . . . 15
12.3 MAP estimation and regularization . . . 15

13 Reinforcement learning **15**
13.1 Value functions and discounted return . . . 16
13.2 Temporal-difference learning . . . 16
13.3 Q-values and control . . . 16
13.4 SARSA and Q-learning . . . 16
13.5 Exploration versus exploitation . . . 17

14 Cross-topic synthesis **17**

Primary sources used **18**

Orientation

These notes now consolidate the full *Machine Intelligence I* lecture arc into one narrative. The goal is not to reproduce every slide verbatim, but to extract the conceptual spine of the course: what kinds of learning problems exist, how neural and probabilistic models are built, how capacity and generalization are controlled, and how sequential decision-making is formalized.

The big arc. The course starts with supervised learning foundations and neural network building blocks, moves through optimization and deep architectures, then turns to capacity control and support vector machines, shifts into probabilistic reasoning and Bayesian inference, and concludes with reinforcement learning as sequential decision-making under uncertainty.

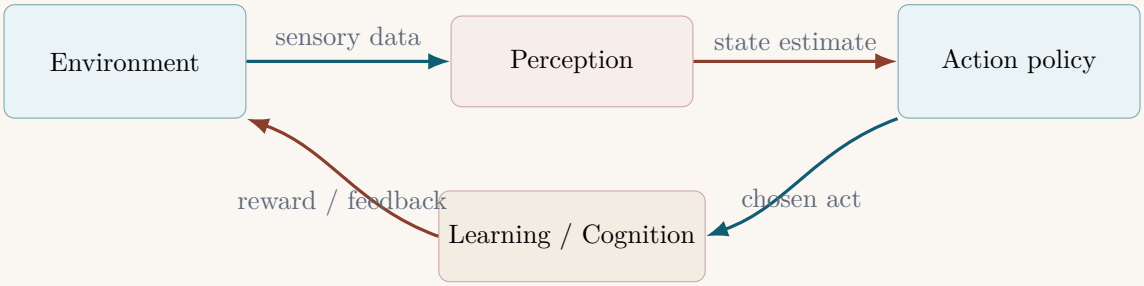
1 Introduction and learning paradigms

Why the course starts here

Machine intelligence is framed as the study of **embedded agents**: systems that perceive the world, build internal representations, and choose actions. The opening contrast is intuitive and motivating:

- **Brains** are strong at pattern recognition, language, inference, and adaptive control.
- **Machines** are strong at exact symbolic manipulation, arithmetic, and brute-force search.

The course goal is to design systems that *learn* useful input–output relationships from data instead of being exhaustively hand-programmed.



Learning as model selection

A machine-learning pipeline can be understood as choosing three things:

1. a **model class**,
2. a **performance measure**, and
3. an **optimization procedure**.

Learning is therefore not just "fit a curve." It is: define a family of candidate hypotheses, define what counts as good performance, and search that family for a good solution.

The three main paradigms

Supervised learning

Learn a mapping from labeled examples $(x^{(\alpha)}, y^{(\alpha)})$. Typical tasks are classification and regression. The central question is: *how should the model predict the target from the input?*

Unsupervised learning

Learn useful structure from inputs alone. Typical goals include clustering, density estimation, and dimensionality reduction. The central question is: *what latent regularities are present in the data?*

Reinforcement learning

Learn behavior from interaction with an environment. Data arrive as trajectories over time rather than IID input–label pairs. The central question is: *which action sequence maximizes expected future reward?*

What matters conceptually

- Supervised learning is about **prediction from examples**.
- Unsupervised learning is about **structure without labels**.
- Reinforcement learning is about **sequential decision-making under delayed feedback**.

2 Connectionist neurons

The basic computational unit

A connectionist neuron is a mathematical abstraction of a biological neuron. Its core computation is

$$y_i = f\left(\sum_j w_{ij} x_j - \theta_i\right),$$

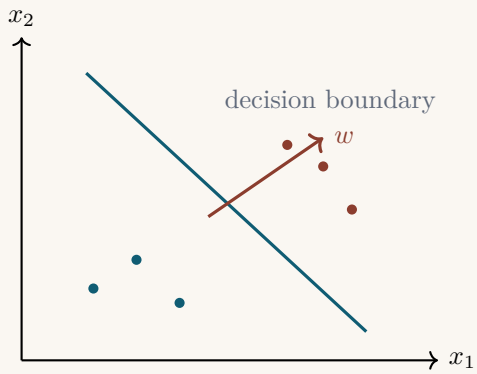
where x_j are inputs, w_{ij} are weights, θ_i is a threshold or bias term, and f is a transfer function.

The neuron first aggregates evidence through a weighted sum and then pushes that quantity through a nonlinearity. This single pattern already supports both representation and decision-making.

Two key intuitions

There are two very useful ways to interpret a connectionist neuron:

1. **Feature detector.** The weight vector selects a direction or pattern in the input space.
2. **Decision boundary.** The neuron defines a hyperplane; the activation indicates which side of that boundary the input falls on.



Transfer functions

Important transfer functions include:

- linear,
- binary threshold,
- logistic / sigmoid,
- tanh,
- ReLU.

Why do we care about the nonlinearity? Because it lets neurons do more than linear regression-like filtering. Nonlinear transfer functions enable classification, richer decision surfaces, and interpretable outputs such as probabilities in appropriate settings.

Geometry to remember

- The weight vector w is normal to the separating hyperplane.
- Changing the bias or threshold shifts the hyperplane without rotating it.
- The sign and magnitude of $w^\top x - \theta$ describe how strongly the input aligns with the neuron's preferred direction.

3 Multilayer perceptrons and backpropagation

Why one layer is not enough

A single linear threshold unit cannot solve non-linearly separable problems such as XOR. A multilayer perceptron (MLP) solves this by stacking multiple nonlinear units. Earlier layers transform the representation; later layers solve an easier problem in the transformed space.

Network classes

The lecture distinguishes two broad classes:

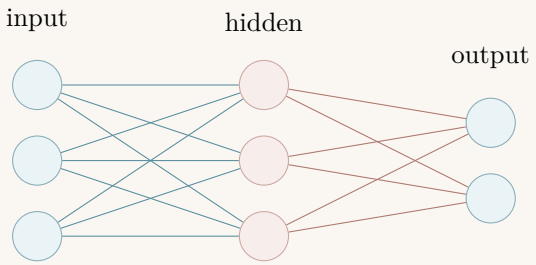
- **Feedforward networks**: information flows from input to output.

- **Recurrent networks**: the graph contains cycles and internal state.

For the early foundations of the course, the emphasis is on feedforward networks.

Universal approximation — with a caveat

An MLP with hidden units can approximate a broad range of functions. But expressive power is not the same as trainability. A network may be flexible enough to represent a solution while still being hard to optimize in practice.



Backpropagation in one sentence

Backpropagation computes how each weight influences the loss by applying the chain rule from the output layer back toward earlier layers.

Training loop

1. Forward pass: compute activations and outputs.
2. Compute the loss.
3. Backward pass: propagate error signals backward.
4. Update weights using the negative gradient.

For any individual parameter, backpropagation answers the question:

> If I change this weight slightly, how much does the final loss change?

That is what makes large layered networks trainable: local derivatives can be composed into global gradients.

Classification link

For multi-class prediction, a standard combination is:

- **softmax** at the output, to convert scores into class probabilities,
- **cross-entropy loss**, to penalize low probability assigned to the true class.

This pairing becomes the modern default for multi-class neural network classification.

4 Optimization, regularization, and validation

Training well is not just about gradients

Once the network architecture exists, the next question is how to train it effectively and how to prevent it from simply memorizing the training data.

Optimization regimes

Batch gradient descent

Uses the full dataset for each gradient step. This gives a stable estimate of the gradient but is often too expensive for large datasets.

Online / stochastic gradient descent

Uses one sample at a time. It is cheap per update but noisy, can wander around minima, and often requires learning-rate annealing.

Mini-batch optimization

This is the practical compromise and the modern standard. It is more stable than pure stochastic updates and much cheaper than full-batch training.

Improving optimization

The lectures emphasize several important refinements:

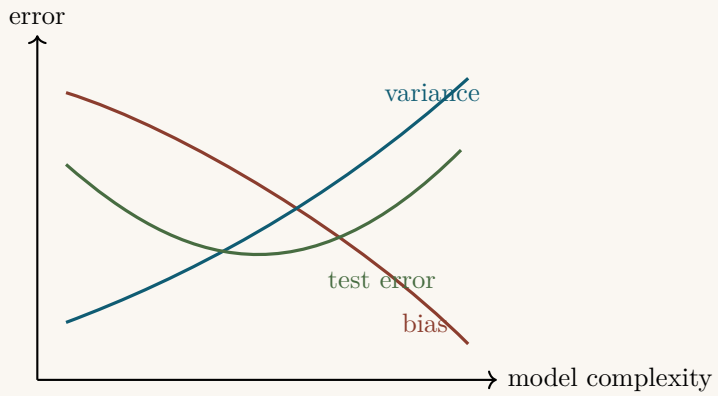
- **Momentum / impulse terms**: smooth updates and maintain motion in consistent downhill directions.
- **Adaptive step sizes**: avoid using a crude fixed learning rate everywhere.
- **Conjugate gradient / line search ideas**: use local curvature information more intelligently.
- **Adam**: combine momentum-like averaging with per-parameter adaptive scaling.

Training error vs. generalization error

A model can fit the training data well and still fail on new data.

- **Training error**: performance on observed samples.
- **Generalization error**: expected performance on new samples from the same data distribution.

The gap between these two quantities is the main reason regularization and validation matter.



Overfitting, underfitting, bias, variance

- **Underfitting**: the model is too simple; both train and test performance are poor.
- **Overfitting**: the model is too flexible; training performance is strong but test performance degrades.
- **Bias**: systematic error from models that are too rigid.
- **Variance**: sensitivity to the particular sampled training set.

As model complexity increases, bias tends to fall while variance tends to rise.

Regularization tools

Weight decay

Add a norm penalty to the objective so that large weights are discouraged. This favors simpler solutions and shrinks parameter values that are not well supported by data.

Early stopping

Monitor validation performance and stop training when it ceases to improve. This acts as an implicit regularizer and can prevent continued fitting of noise.

Validation discipline

Keep training, validation, and test roles conceptually separate: training fits parameters, validation chooses hyperparameters or stopping point, and the test set is reserved for final unbiased evaluation.

Cross-validation

Average validation performance across folds when data are limited to obtain a more reliable estimate of out-of-sample performance.

5 Deep learning foundations

What changed, and why

Deep learning is not a different philosophy from earlier machine learning topics. It extends the same pipeline — representation, objective, optimization, and regularization — but at greater representational depth and practical scale.

Why deep networks originally looked difficult

The course notes emphasize that deep multilayer perceptrons were once seen as hard to train because of:

- vanishing gradients with sigmoid-like nonlinearities,
- long convergence times,

- large parameter counts,
- datasets that were too small,
- insufficient computational power.

This is why shallow methods and support vector machines dominated for a period.

Why deep learning came back

Several things improved together:

1. larger datasets,
2. far more compute,
3. improved tricks of the trade,
4. better representations learned across many layers.

The key idea is that depth allows successive layers to build increasingly abstract features.

Deep representations

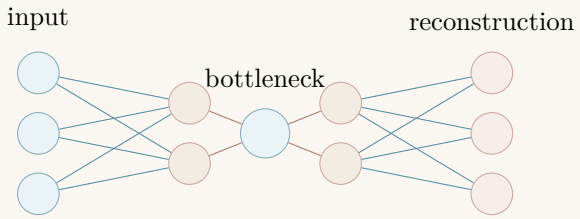
A deep network can form a hierarchy:

- early layers capture simple local structure,
- middle layers combine those into motifs or parts,
- later layers represent task-relevant abstractions.

This helps explain why deep models can outperform shallow alternatives even when both are expressive in principle.

Autoencoders and pre-training

An autoencoder learns to compress and reconstruct data. The bottleneck forces a compact internal representation, which can be useful for unsupervised pre-training or initialization before fine-tuning on a downstream task.



Practical regularization in deep networks

- **Data augmentation**: synthesize additional plausible inputs by transforming existing examples.
- **Dropout**: randomly deactivate units during training to discourage co-adaptation and improve robustness.

6 Recurrent neural networks

Learning from sequences instead of IID examples

In sequence problems, samples inside one sequence are not IID. The prediction at time step t depends on what came before, so the model must retain some form of memory. A fixed-window feedforward approximation conditions only on a recent context,

$$P(y_t \mid y_{t-k}, \dots, y_{t-1}),$$

but this forces us to choose a finite horizon in advance. Recurrent neural networks replace the explicit window by a latent state that is updated over time.

Core RNN dynamics

A basic recurrent network processes one input at a time:

$$s_t = f(Ux_t + Ws_{t-1} + b_s), \qquad y_t = g(Vs_t + b_y).$$

Here s_t is the hidden state, U maps the current input into state space, W carries forward the previous state, and V maps state to output.

Intuition. The hidden state s_t is a compressed memory of the past. Instead of remembering every earlier input explicitly, the network learns what should be preserved and what can be forgotten.

Backpropagation through time

For a sequence dataset, the cost accumulates over time,

$$E = \frac{1}{p} \sum_{\alpha=1}^{p} \frac{1}{n_\alpha} \sum_{t=1}^{n_\alpha} e^{(\alpha,t)},$$

so training unfolds the recurrence across time and applies backpropagation through time (BPTT).

Intuition. An RNN is a deep network in disguise, where the depth dimension is time. BPTT sends error signals backward through earlier hidden states that influenced the present prediction.

Why long-term dependencies are hard

If we linearize the recurrence, repeated application of the recurrent matrix gives

$$s_t = W^t s_0.$$

The same repeated multiplication appears in the gradient, which leads to the classic problems of vanishing and exploding gradients:

- if the effective gain is too small, information and gradients decay exponentially;
- if it is too large, activity and gradients blow up.

Intuition. Tiny shrinkage at each step eventually becomes total forgetting; tiny amplification eventually becomes instability.

Architectural fixes

The lectures motivate several responses:

- **Echo-state networks**: keep the recurrent reservoir fixed and only train the readout.
- **Leaky units**: introduce slower state averaging,

$$\mu_t = \alpha\mu_{t-1} + (1-\alpha)s_{t-1}, \qquad 0 < \alpha < 1.$$

- **LSTM-style gating**: maintain a protected cell state and control writing, forgetting, and reading through gates,

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad s_t = o_t \odot \tanh(c_t).$$

Intuition. Gating creates a memory line that can persist across many time steps without forcing every intermediate nonlinear transformation to preserve the signal.

7 Radial basis function networks

Local approximation rather than global hidden features

A radial basis function (RBF) network predicts with a linear combination of localized basis functions:

$$y(x) = \sum_{i=1}^{M} w_i\, \phi_i(x), \qquad \phi_i(x) = \exp\left(-\frac{\|x - t_i\|^2}{2\sigma_i^2}\right).$$

Each hidden unit is centered at a prototype t_i with width σ_i.

Intuition. Each basis function fires only near its own center, so the network behaves like a patchwork of local experts rather than a single global feature hierarchy.

Why RBFs are attractive

RBF networks often learn quickly because only nearby basis functions matter strongly for a given input. But they pay for this locality with poor scaling in high dimension: to cover an N-dimensional space with roughly M basis functions per dimension requires on the order of M^N local regions.

Intuition. Local approximation is powerful when the data lives in low-dimensional or clustered structure, but expensive when the ambient space is large.

Two-step learning procedure

A practical learning pipeline is:

1. choose centers t_i by unsupervised learning,
2. choose widths σ_i heuristically,
3. solve the output weights w_i by supervised linear regression.

Centers are often learned by online k-means:

$$i = \arg\min_j \|x^{(\alpha)} - t_j\|, \qquad \Delta t_i = \eta_t (x^{(\alpha)} - t_i).$$

A standard width heuristic is

$$\sigma_i = \lambda \min_{j \neq i} \|t_i - t_j\|,$$

with λ chosen to ensure enough overlap between neighboring basis functions.

Closed-form output learning

Once the hidden features are fixed, the model is linear in w. With design matrix entries

$$\Phi_{\alpha i} = \phi_i(x^{(\alpha)}),$$

the least-squares solution satisfies the normal equations

$$\Phi^\top \Phi\, w = \Phi^\top y_T, \qquad w = (\Phi^\top \Phi)^{-1} \Phi^\top y_T$$

when the inverse exists.

Intuition. The hard nonlinear part is delegated to the basis functions; the final fitting step is just linear regression in the transformed feature space.

Normalization and regularization

A normalized RBF layer uses

$$\psi_i(x) = \frac{\phi_i(x)}{\sum_j \phi_j(x)}, \qquad y(x) = \sum_i w_i \psi_i(x),$$

which turns the hidden layer into a soft interpolation over nearby prototypes. If the least-squares problem is ill-conditioned, ridge regularization gives

$$w = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top y_T.$$

Intuition. Regularization picks smoother functions when many different local interpolants fit the same training data.

8 Statistical learning theory

Why low training error is not enough

The central question of statistical learning theory is: when does minimizing training error actually produce a predictor that generalizes? For a predictor $y(x; w)$ and loss $e(y, x; w)$, the true risk is

$$R(w) = \int P(x, y)\, e(y, x; w)\, dx\, dy,$$

while the empirical risk is

$$R_{\text{emp}}(w) = \frac{1}{p} \sum_{i=1}^{p} e(y_i, x_i; w).$$

Empirical risk minimization (ERM) chooses

$$w_p = \arg\min_w R_{\text{emp}}(w).$$

Intuition. ERM replaces an unknown expectation by a sample average. That replacement is only justified if the sample average is uniformly close to the true expectation across the whole model class.

Capacity and the growth function

Capacity is captured by the number of distinct labelings that a hypothesis class can realize on p points. The growth function measures the worst-case logarithmic count of those labelings. If capacity is too high, the class can fit arbitrary labels and low training error becomes uninformative.

For linear classifiers in general position, Cover's counting argument gives the number of separable assignments:

$$C(p, N) = 2 \sum_{k=0}^{N-1} \binom{p-1}{k}.$$

VC dimension

The VC dimension d_{VC} is the largest number of points that can be shattered by the model class. A key learning-theory message is that finite VC dimension makes generalization possible. Heuristically, generalization bounds have the form

$$R(w) \le R_{\text{emp}}(w) + C(p, d_{\text{VC}}),$$

where the second term penalizes model capacity.

Intuition. Good learning requires both fit and restraint: a flexible enough class to represent the solution, but not so flexible that fitting the sample is trivial.

Structural risk minimization

Structural risk minimization (SRM) replaces "minimize training error in one fixed class" with "balance training fit against complexity across nested classes." This is the conceptual bridge from statistical learning theory to support vector machines.

9 Support vector machines

Margin maximization as capacity control

For a linear classifier

$$f(x) = w^\top x + b, \qquad y(x) = \text{sign}(f(x)),$$

the canonical normalization is chosen so that

$$\min_i y_i(w^\top x_i + b) = 1.$$

Then the distance of the closest point to the hyperplane is proportional to $1/\|w\|$, so maximizing the margin is equivalent to minimizing $\frac{1}{2}\|w\|^2$.

Intuition. Among all separating hyperplanes, choose the one with the widest safety buffer. A larger margin means lower effective capacity and better robustness to perturbations.

Hard-margin primal problem

For linearly separable data, the SVM solves

$$\min_{w,b} \ \frac{1}{2}\|w\|^2$$

subject to

$$y_i(w^\top x_i + b) \geq 1, \qquad i = 1, \ldots, p.$$

Dual formulation and support vectors

Introducing Lagrange multipliers $\alpha_i \geq 0$ yields the dual problem

$$\max_{\alpha} \ \sum_{i=1}^{p} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{p} \alpha_i \alpha_j y_i y_j \, x_i^\top x_j$$

subject to

$$\sum_{i=1}^{p} \alpha_i y_i = 0, \qquad \alpha_i \geq 0.$$

The optimal classifier has the form

$$w^* = \sum_{i=1}^{p} \alpha_i^* y_i x_i, \qquad y(x) = \operatorname{sign}\left(\sum_{i=1}^{p} \alpha_i^* y_i x_i^\top x + b^* \right).$$

Only points with $\alpha_i^* > 0$ remain in the final solution; these are the support vectors.

Intuition. Most training points disappear. The classifier is determined only by the points that sit on the margin and therefore constrain the decision boundary.

Soft margins and kernels

Real data are usually not perfectly separable, so slack variables $\xi_i \geq 0$ produce the soft-margin objective

$$\min_{w,b,\xi} \ \frac{1}{2}\|w\|^2 + \frac{C}{p} \sum_{i=1}^{p} \xi_i$$

subject to

$$y_i(w^\top x_i + b) \geq 1 - \xi_i.$$

The hyperparameter C controls the trade-off between a wide margin and few violations.

In the dual, data appear only through inner products. Replacing $x_i^\top x_j$ by a kernel $k(x_i, x_j)$ gives the kernel SVM:

$$y(x) = \operatorname{sign}\left(\sum_{i \in SV} \alpha_i y_i \, k(x_i, x) + b \right).$$

Common choices include the polynomial kernel and the Gaussian kernel,

$$k(x, x') = (x^\top x' + 1)^d, \qquad k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right).$$

Intuition. The kernel trick keeps optimization linear in feature space while making the decision boundary nonlinear in the original input space.

10 Uncertainty and inference

Probability as rational belief under incomplete information

This lecture block treats probability as a calculus of plausible reasoning. A proposition H is assigned a degree of belief $P(H) \in [0, 1]$, and those beliefs must obey the probability axioms to remain coherent.

Inference by marginalization and conditioning

If x is a query and e is observed evidence, then

$$P(x \mid e) = \frac{P(x, e)}{P(e)}.$$

If there are hidden variables y, inference requires summing them out:

$$P(x \mid e) = \alpha \sum_y P(x, e, y), \qquad \alpha^{-1} = \sum_{x,y} P(x, e, y).$$

Bayes' rule follows from the product rule:

$$P(Y \mid X) = \frac{P(X \mid Y)P(Y)}{P(X)}.$$

Intuition. Inference means "sum over what you do not know, then renormalize over what remains."

Conditional independence

The main route to tractable probabilistic reasoning is conditional independence:

$$X \perp Y \mid Z \qquad \Longleftrightarrow \qquad P(X, Y \mid Z) = P(X \mid Z)P(Y \mid Z).$$

This is the idea that once Z is known, learning Y tells us nothing further about X.

A classic simplification is Naive Bayes,

$$P(X_1, \ldots, X_{N-1}, Y) = P(Y) \prod_{i=1}^{N-1} P(X_i \mid Y),$$

which drastically reduces complexity by assuming the observed variables become independent once the latent cause Y is known.

11 Bayesian networks

Encoding probabilistic structure with directed acyclic graphs

A Bayesian network is a directed acyclic graph whose nodes are random variables and whose edges encode direct statistical dependencies. Its defining factorization is

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P\big(X_i \mid \mathrm{pa}(X_i)\big),$$

where $\mathrm{pa}(X_i)$ denotes the parents of X_i.

Intuition. Instead of storing one huge joint table, we store one local conditional model per variable given only its direct causes.

Graph structure and independence

The graph determines which variables become independent when evidence is observed. Chains and forks are blocked by conditioning on the middle or common-cause node, while colliders behave oppositely: conditioning on a common effect can create dependence.

Markov blankets

For a node A, the Markov blanket consists of its parents, its children, and the other parents of its children. Given the Markov blanket,

$$A \perp \text{all other nodes} \mid \mathrm{MB}(A).$$

Intuition. Once you know what directly influences A, what A influences, and who competes with A in explaining those effects, the rest of the graph becomes irrelevant for predicting A.

Inference in Bayesian networks

Inference still amounts to multiplying local factors and summing out hidden variables,

$$P(x \mid e) \propto \sum_{y} \prod_{i} P(X_i \mid \mathrm{pa}(X_i)).$$

The graph structure makes that computation much more efficient than naive enumeration and motivates algorithms based on elimination orderings and message passing.

12 Bayesian inference and neural networks

From one best model to posterior-weighted prediction

The Bayesian viewpoint starts from a generative account of data. For supervised data $z = (x, y)$, we factor the joint distribution as

$$p(z) = p(y \mid x)\, p(x).$$

For regression, a simple generative model is

$$y = y^*(x) + \eta,$$

where $y^*(x)$ is the underlying deterministic relation and η is noise.

Bayesian model selection

Competing models are compared by Bayes' rule,

$$P(M \mid E) = \frac{P(E \mid M)P(M)}{P(E)}.$$

The posterior favors models that both explain the evidence well and were plausible a priori.

Intuition. A good model is not merely one that can fit the data somewhere in parameter space; it is one under which the observed data was genuinely likely.

Bayesian prediction

Prediction averages over model uncertainty rather than committing to a single best model:

$$P(y \mid x, D) = \int P(y \mid x; w)\, P(w \mid D)\, dw.$$

For discrete model families, this becomes a posterior-weighted committee over models.

MAP estimation and regularization

For neural networks, if the likelihood and prior take exponential forms,

$$P(Y \mid X; w) \propto \exp\bigl(-\beta E_T(w)\bigr), \qquad P(w) \propto \exp\bigl(-\alpha E_R(w)\bigr),$$

then the posterior is proportional to

$$P(w \mid X, Y) \propto \exp\bigl(-\beta E_T(w) - \alpha E_R(w)\bigr).$$

Therefore the MAP estimate solves a regularized optimization problem,

$$w^* = \arg\min_w \bigl(E_T(w) + \alpha' E_R(w)\bigr).$$

For quadratic regularization, this is weight decay.

Intuition. Regularization is not just a computational hack; it is equivalent to imposing prior beliefs on parameters.

13 Reinforcement learning

Sequential decision-making under delayed reward

Reinforcement learning studies how an agent should act in an environment when rewards can be delayed. The standard formalism is the Markov decision process (MDP), consisting of states x, actions a, transition probabilities $P(x' \mid x, a)$, rewards $r(x, a)$, and a policy $\pi(a \mid x)$.

Value functions and discounted return

The value of a policy π at state x is the expected discounted return,

$$V^\pi(x) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(x_t, a_t) \;\middle|\; x_0 = x,\ a_t \sim \pi(\cdot \mid x_t)\right], \qquad \gamma \in [0, 1).$$

The Bellman equation expresses this recursively:

$$V^\pi(x_i) = \sum_k \pi(a_k \mid x_i) r(x_i, a_k) + \gamma \sum_{j,k} \pi(a_k \mid x_i) P(x_j \mid x_i, a_k) V^\pi(x_j).$$

Intuition. A state is valuable if it gives reward now or tends to lead to valuable states later.

Temporal-difference learning

Monte Carlo evaluation waits for full returns. Temporal-difference learning bootstraps from the next estimate:

$$\tilde{V}_{t+1}(x_t) = \tilde{V}_t(x_t) + \eta\Big[r_t + \gamma \tilde{V}_t(x_{t+1}) - \tilde{V}_t(x_t)\Big].$$

The bracketed term is the TD error.

Intuition. TD learning updates immediately by treating the next prediction as a temporary target, rather than waiting until the end of the episode.

Q-values and control

For action selection we need values of state-action pairs:

$$Q^\pi(x_i, a_k) = r(x_i, a_k) + \gamma \sum_j P(x_j \mid x_i, a_k) V^\pi(x_j), \qquad V^\pi(x) = \sum_a \pi(a \mid x) Q^\pi(x, a).$$

A greedy policy chooses the action with largest Q-value.

SARSA and Q-learning

SARSA is on-policy:

$$Q_{t+1}(x_t, a_t) = Q_t(x_t, a_t) + \eta\Big[r_t + \gamma Q_t(x_{t+1}, a_{t+1}) - Q_t(x_t, a_t)\Big].$$

Q-learning is off-policy:

$$Q_{t+1}(x_t, a_t) = Q_t(x_t, a_t) + \eta\Big[r_t + \gamma \max_{a'} Q_t(x_{t+1}, a') - Q_t(x_t, a_t)\Big].$$

Intuition. SARSA learns the value of the policy the agent is actually following; Q-learning learns the value of acting optimally from the next step onward even while the behavior policy remains exploratory.

Exploration versus exploitation

The agent must balance trying uncertain actions against exploiting currently good ones. Standard choices include ε-greedy exploration and softmax action selection,

$$\pi(a \mid x) = \frac{\exp(\beta Q(x, a))}{\sum_{a'} \exp(\beta Q(x, a'))}.$$

Intuition. Without exploration, the agent can get stuck with a locally good but globally poor strategy; without exploitation, it never benefits from what it has learned.

14 Cross-topic synthesis

One storyline, not isolated lecture blocks

The complete course fits together as one argument:

1. **Learning foundations** define model classes, objectives, and training procedures.
2. **Neural network models** build expressive function approximators from neurons, hidden layers, recurrent state, and localized basis functions.
3. **Learning theory** explains when low training error can be trusted and why capacity control matters.
4. **Large-margin methods** turn those ideas into concrete optimization principles via support vector machines.
5. **Probabilistic reasoning** handles uncertainty through structured distributions, Bayesian updating, and model averaging.
6. **Reinforcement learning** extends prediction to sequential control under delayed reward.

If you only remember twelve things:

1. Machine learning can be framed as model selection.
2. A neuron is a weighted sum followed by a nonlinearity.
3. Hidden state is the key to sequence modeling.
4. RBF networks approximate locally, not globally.
5. Training error and generalization error are not the same thing.
6. Capacity control is the central theme of statistical learning theory.
7. Margins connect geometry to generalization.
8. SVMs solve structural risk minimization in optimization form.
9. Bayesian reasoning updates prior beliefs with evidence.
10. Graph structure can make high-dimensional inference tractable.
11. Regularization can be interpreted as a prior over parameters.
12. Reinforcement learning is value-based reasoning over future reward, not just immediate reward.

Primary sources used

- `Lecture Slides/11-Introduction.pdf`
- `Lecture Slides/12-Connectionist Neurons.pdf`
- `Lecture Slides/13-Multilayer Perceptrons.pdf`
- `Lecture Slides/14-Additional Topics.pdf`
- `Lecture Slides/15-Deep Learning.pdf`
- `Lecture Slides/16-Recurrent Neural Networks.pdf`
- `Lecture Slides/17-Radial Basis Function Networks.pdf`
- `Lecture Slides/21-Elements of Statistical Learning Theory.pdf`
- `Lecture Slides/22-Support Vector Machines.pdf`
- `Lecture Slides/31-Uncertainty and Inference.pdf`
- `Lecture Slides/32-Bayesian Networks.pdf`
- `Lecture Slides/33-Bayesian Inference and Neural Networks.pdf`
- `Lecture Slides/41-Reinforcement Learning - Valuation.pdf`
- `Lecture Slides/42-Reinforcement Learning - Policy Learning.pdf`
- Existing Obsidian lecture and tutorial notes for the corresponding topics